

第 12 讲实验设计文档

不用 PyTorch，从头验证一个两层 MLP 的完整 Forward

实验目标	把“神经网络 Forward”从抽象概念变成可以逐步核对的计算过程，并验证单样本、Batch、参数扰动三种情况下的行为。
------	--

实验问题

- 给定固定参数后，NumPy 能否严格按照手算公式完成两层 MLP Forward？
- Batch 一次计算是否与逐样本计算完全等价？
- 只改变一个 Weight 时，影响是否沿对应计算路径向后传播到最终输出？
- 仅有 Forward 时，模型是否发生学习？

1. 假设与可验证预测

假设	预测
H1：矩阵实现与手算一致	第一个隐藏神经元手算值应与 $z_1[0]$ 完全一致
H2：Batch 只是批量执行同一函数	Batch 结果应与逐样本 stack 后完全一致
H3：参数影响沿连接传播	只改 $W_1[0,0]$, z_1 首先只改变第 0 个隐藏单元
H4：Forward 不等于学习	执行结束后没有任何自动参数更新步骤

变量设计

- 自变量：输入样本、是否使用 Batch、某一个 Weight 的数值。
- 因变量： z_1 、 a_1 、 y 的数值和 Shape。
- 控制变量：其他所有 Weight/Bias、网络结构、Activation。

2. 实验环境与文件

- Python 3.x
- NumPy
- 不需要 GPU
- 不需要 PyTorch
- 脚本：lesson12_numpy_mlp_forward_experiment.py

为什么本讲故意不用 PyTorch？

因为当前目标是建立最低闭环能力：你需要先亲自看见矩阵乘法、Bias 和 ReLU 是怎样组成 Forward 的。等进入 PyTorch 阶段时，才知道框架替你封装了什么。

运行方式	在终端进入文件目录后执行：python lesson12_numpy_mlp_forward_experiment.py
------	--

3. 网络与参数设置

$3 \rightarrow 4 \rightarrow 2$

参数	Shape	用途
W_1	(3,4)	第一层权重
b_1	(4,)	第一层偏置
W_2	(4,2)	第二层权重
b_2	(2,)	第二层偏置

总参数量：

$3 \times 4 + 4 + 4 \times 2 + 2 = 26$

单样本输入：

$x = [1, 2, -1]$

4. 实验步骤

实验 1：检查 Shape 与参数数量

先不计算输出，只打印 W_1 、 b_1 、 W_2 、 b_2 的 Shape 和总元素数量。目的是建立“结构先于数值”的习惯。

实验 2：单样本完整 Forward

依次打印 x 、 z_1 、 a_1 、 y ，并在每一步同时打印 Shape。

实验 3：手工核对一个隐藏神经元

只核对 $z_1[0]$ ，用逐项乘加确认矩阵乘法不是黑盒。

实验 4：Batch Forward

构造 3 个样本组成 X .shape=(3,3)，一次得到 Y .shape=(3,2)。

4. 实验步骤（续）

实验 5：Batch 与逐样本对照

将每个样本单独 forward，再 stack，与一次 Batch Forward 做差。若实现正确，最大绝对误差应为 0（或仅浮点误差）。

实验 6：单 Weight 扰动

只修改 $W_1[0,0] += 1$ ，比较修改前后的 z_1 与 y 。该实验展示一个局部参数如何沿计算图影响后续节点。

实验 7：概念检查

观察脚本中是否存在 Loss、Backward、Optimizer 或参数更新代码。没有，因此当前只是 Forward，不是 Training。

5. 预期结果

`z1 = [ 3.1 -1.2 -1.7 0.5]`
`a1 = [3.1 0.0 0.0 0.5]`
`y = [ 3.3 -1.35]`

Batch
`Y = [[ 3.3 -1.35]`
 `[-2. 3.2 ]`
 `[ 1.2 2.75]]`

Batch vs = 0.0

参数扰动预期	只改 W_1 [0,0] 后, z_1 的变化应先表现为 [1,0,0,0] ; 随后通过 W_2 传播为 y 的变化 [1,-0.5]。
--------	---

6. 如何解释结果

- 手算 = NumPy：说明你对矩阵乘法、Bias 和 Shape 的理解与程序一致。
- Batch = 逐样本：说明批量计算不是不同规则，只是把同一个函数并行/向量化执行。
- 局部 Weight 扰动产生路径化影响：说明 Computational Graph 的“依赖关系”可以通过实验直接观察。
- 没有参数更新：说明 Forward 与 Training 必须严格区分。

科研训练意义

这不是为了做一个“很厉害”的模型，而是在训练一种科研习惯：先明确假设，再设计最小实验，再观察中间变量，而不是只看最终输出。

7. 常见失败与排错

问题	可能原因	检查方法
matmul Shape 报错	W 的输入维与 x 最后一维不一致	先写 Shape，再做 @
ReLU 后 Shape 变了	误用了聚合/索引操作	确认使用 np.maximum(0,x)
Batch 与逐样本不同	参数、顺序或 stack 维度不一致	逐个打印中间 z_1/a_1
结果和手算不同	权重索引方向理解错误	确认列代表隐藏神经元
排错原则	先查 Shape，再查单个元素，再查整层。不要一上来盯着最终 y 猜原因。	

8. 实验记录模板

建议你运行代码后，用下面模板自己填一次：

记录项	你的结果
$W_1/b_1/W_2/b_2$ Shape	_____
总参数量	_____
单样本 z_1	_____
单样本 a_1	_____
单样本 y	_____
Batch 输出 Shape	_____
Batch vs 逐样本最大误差	_____
扰动后 z_1 变化	_____
扰动后 y 变化	_____

完成标准	不要只“运行成功”。至少能解释为什么每个结果应该是那个 Shape，以及参数扰动为什么只从特定路径开始传播。
------	--